K. J. Somaiya School of Engineering, Mumbai-77

(Somaiya Vidyavihar University)

Department of Computer Engineering

Batch: B2	Roll No.: 16010124107
Experiment / assignment / tutorial No. 7	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

Title: Indexing - Create Index and observe the evaluation statistics of query execution with and without Index

Objective: Implement indexing to improve query execution plans

Expected Outcome of Experiment:
CO 4: Analyse Advanced Database Concepts like indexing, hashing, query processing, query optimization, normalization.

Books/ Journals/ Websites referred:

1. Dr. P.S. Deshpande, SQL and PL/SQL for Oracle 10g.Black book, Dreamtech Press
2. www.db-book.com
3. Korth, Slberchatz, Sudarshan : "Database Systems Concept", 5th Edition , McGraw Hill
4. Elmasri and Navathe,"Fundamentals of database Systems", 4th Edition,PEARSON Education.

Resources used: PostgreSQL

Theory:

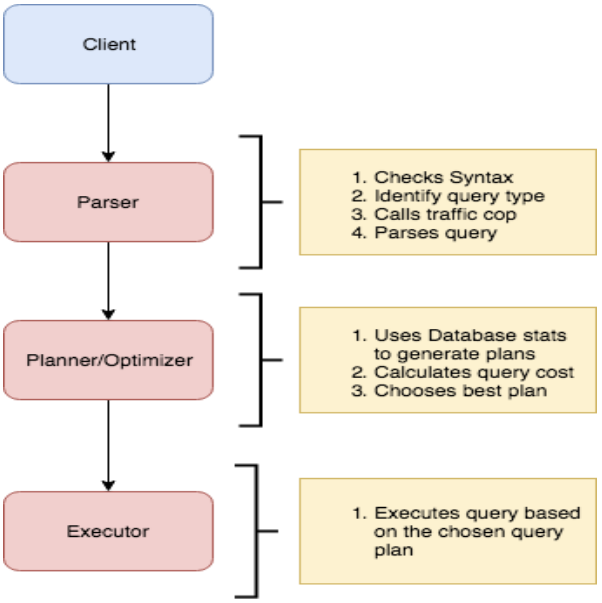
A database index is a data structure that improves the speed of operations in a table. Indexes can be created using one or more columns, providing the basis for both rapid random lookups and efficient ordering of access to records.

While creating index, it should be taken into consideration which all columns will be used to make SQL queries and create one or more indexes on those columns.

To add an index for a column or a set of columns, you use the `CREATE INDEX` statement as follows:

```
CREATE INDEX index_name ON table_name (column_list)
```

Query life cycle



Planner and Executor:

The planner receives a query tree from the rewriter and generates a (query) plan tree that can be processed by the executor most effectively.

The planner in Database is based on pure cost-based optimization -

EXPLAIN command:

This command displays the execution plan that the PostgreSQL/MySQL planner generates for the supplied statement. The execution plan shows how the table(s) referenced by the statement will be scanned — by plain sequential scan, index scan, etc. — and if multiple tables are referenced, what join algorithms will be used to bring together the required rows from each input table.

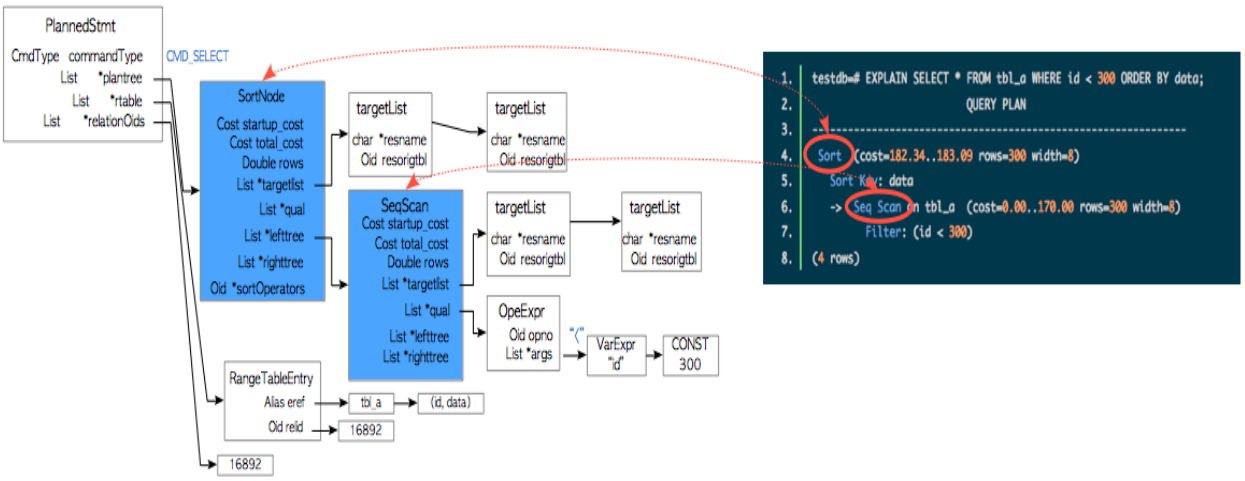
As in the other RDBMS, the EXPLAIN command in Database displays the plan tree itself. A specific example is shown below:-

Database: testdb=#

1. EXPLAIN SELECT * FROM tbl_a WHERE id < 300 ORDER BY data;
2. QUERY PLAN

```
3. ----------------------------------------------------------------
4. Sort (cost=182.34..183.09 rows=300 width=8)
5. Sort Key: data
6. -> Seq Scan on tbl_a (cost=0.00..170.00 rows=300 width=8)
7. Filter: (id < 300)
8. (4 rows)
```

A simple plan tree and the relationship between the plan tree and the result of the EXPLAIN command in PostgreSQL.



Nodes

The first thing to understand is that each indented block with a preceeding "->" (along with the top line) is called a node. A node is a logical unit of work (a "step" if you will) with an associated cost and execution time. The costs and times presented at each node are cumulative and roll up all child nodes.

Cost:

It is not the time but a concept designed to estimate the cost of an operation. The first number is start-up cost (cost to retrieve first record) and the second number is the cost incurred to process entire node (total cost from start to finish).

Cost is a combination of 5 work components used to estimate the work required: sequential fetch, non-sequential (random) fetch, processing of row, processing operator (function), and processing index entry.

Rows are the approximate number of rows returned when a specified operation is performed.
(In the case of select with where clause rows returned is
Rows = cardinality of relation * selectivity)
Width is an average size of one row in bytes**.**

Explain Analyze command:

The EXPLAIN ANALYZE option causes the statement to be actually executed, not only planned. Then actual run time statistics are added to the display, including the total elapsed time expended within each plan node (in milliseconds) and the total number of rows it actually returned. This is useful for seeing whether the planner's estimates are close to reality.

Ex: EXPLAIN (ANALYZE) SELECT * FROM foo;

```
QUERY PLAN
— Seq Scan on foo (cost=0.00..18334.10 rows=1000010 width=37) (actual time=0.012..61.524
rows=1000010 loops=1)
Total runtime: 90.944 ms
(2 rows)
```

The command displays the following additional parameters:

- **actual time** is the actual time in milliseconds spent to get the first row and all rows, respectively.
- **rows** is the actual number of rows received with Seq Scan.
- **loops** is the number of times the Seq Scan operation had to be performed.
- **Total runtime** is the total time of query execution.

Query plans for select with where clause can be sequential scan, Index Scan, Index only Scan, Bitmap Index Scan etc.

Query plans for joins are Nested loop join, Hash join, Merge join etc.

Indexing: `CREATE INDEX` constructs an index on the specified column(s) of the specified relation, which can be a table or a materialized view. Indexes are primarily used to enhance database performance (though inappropriate use can result in slower performance).

Syntax

```
 CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ [ IF NOT EXISTS ]
name ] ON [ ONLY ] table_name [ USING method ]

    ( { column_name | ( expression ) } [ COLLATE collation ] [
opclass [ ( opclass_parameter = value [, ... ] ) ] ] [ ASC |
DESC ] [ NULLS { FIRST | LAST } ] [, ...] )

    [ INCLUDE ( column_name [, ...] ) ]

    [ NULLS [ NOT ] DISTINCT ]

    [ WITH ( storage_parameter [= value] [, ... ] ) ]

    [ TABLESPACE tablespace_name ]

    [ WHERE predicate ]
```

example

explain Analyse select * from std2 where branch ='ext'

create index dept on std2(Branch)

select * from std2

explain Analyse select * from std2 where branch ='ext'

drop index dept

Implementation Screenshots :

Comprehend how indexes improves the performance of query applied for your database . Demonstrate for the following types of query on your database

a. **Simple select query**

EXPLAIN ANALYZE
SELECT * FROM customer;

Showing rows: 1 to 4

	QUERY PLAN text
1	Seq Scan on customer (cost=0.00..10.80 rows=80 width=884) (actual time=0.073..0.079 rows=10.00 loop...
2	Buffers: shared hit=1
3	Planning Time: 0.278 ms
4	Execution Time: 0.143 ms

b. Select query with where clause

EXPLAIN ANALYZE
SELECT * FROM customer WHERE name = 'Rahul';

CREATE INDEX idx_customer_name ON customer(name);

EXPLAIN ANALYZE
SELECT * FROM customer WHERE name = 'Rahul';

Showing rows: 1 to

Add row
Alt Shift A

	QUERY PLAN
	n on customer (cost=0.00..1.12 rows=1 width=884) (actual time=0.030..0.034 rows=1.00 loop...
2	Filter: ((name)::text = 'Rahul'::text)
3	Rows Removed by Filter: 9
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=4 read=1
7	Planning Time: 2.353 ms
8	Execution Time: 0.060 ms

c. Select query with order by query

EXPLAIN ANALYZE
SELECT * FROM Accounts ORDER BY balance;

CREATE INDEX idx_accounts_balance ON Accounts(balance);

EXPLAIN ANALYZE
SELECT * FROM Accounts ORDER BY balance;

Data Output Messages Notifications

Showing rows: 1 to 10

	QUERY PLAN text
1	Sort (cost=1.27..1.29 rows=10 width=304) (actual time=0.057..0.058 rows=10.00 loops=1)
2	Sort Key: balance
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=1
5	-> Seq Scan on accounts (cost=0.00..1.10 rows=10 width=304) (actual time=0.044..0.045 rows=10.00 loop...
6	Buffers: shared hit=1
7	Planning:
8	Buffers: shared hit=5 read=1
9	Planning Time: 1.777 ms
10	Execution Time: 0.081 ms

d. Select query with JOIN

```
EXPLAIN ANALYZE
SELECT C.name, A.balance
FROM customer C
JOIN Accounts A ON C.acc_no = A.acc_no;

CREATE INDEX idx_customer_acc ON customer(acc_no);
CREATE INDEX idx_accounts_acc ON Accounts(acc_no);

EXPLAIN ANALYZE
SELECT C.name, A.balance
FROM customer C
JOIN Accounts A ON C.acc_no = A.acc_no;
```

Showing rows: 1 to 14

	QUERY PLAN text
1	Hash Join (cost=1.23..2.36 rows=10 width=226) (actual time=0.168..0.185 rows=10.00 loops=1)
2	Hash Cond: ((c.acc_no)::text = (a.acc_no)::text)
3	Buffers: shared hit=2
4	-> Seq Scan on customer c (cost=0.00..1.10 rows=10 width=336) (actual time=0.029..0.035 rows=10.00 loops=1)
5	Buffers: shared hit=1
6	-> Hash (cost=1.10..1.10 rows=10 width=126) (actual time=0.040..0.040 rows=10.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on accounts a (cost=0.00..1.10 rows=10 width=126) (actual time=0.019..0.024 rows=10.00 loo...
10	Buffers: shared hit=1
11	Planning:
12	Buffers: shared hit=17 read=2

Total rows: 14 Query complete 00:00:00.162

e. Select query with aggregation

```
EXPLAIN ANALYZE
SELECT acc_no, SUM(amt)
FROM transaction
GROUP BY acc_no;

CREATE INDEX idx_transaction_acc ON transaction(acc_no);

EXPLAIN ANALYZE
SELECT acc_no, SUM(amt)
FROM transaction
GROUP BY acc_no;
```

Showing rows: 1 to 10

	QUERY PLAN text
1	HashAggregate (cost=1.15..1.25 rows=10 width=126) (actual time=0.028..0.031 rows=10.00 loops=1)
2	Group Key: acc_no
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=1
5	-> Seq Scan on transaction (cost=0.00..1.10 rows=10 width=126) (actual time=0.013..0.014 rows=10.00 loo...
6	Buffers: shared hit=1
7	Planning:
8	Buffers: shared hit=8 read=1
9	Planning Time: 1.187 ms
10	Execution Time: 0.065 ms

```
CREATE TABLE users (
   id SERIAL PRIMARY KEY,
   name TEXT,
   email TEXT
);


INSERT INTO users (name, email) VALUES
('Alice', 'alice@example.com'),
('Bob', 'bob@example.com'),
('Charlie', 'charlie@example.com');


EXPLAIN ANALYZE  SELECT * FROM users WHERE name = 'Alice';
```

Data Output Messages Notifications

Showing rows: 1

	QUERY PLAN text
1	Seq Scan on users (cost=0.00..20.62 rows=4 width=68) (actual time=0.029..0.031 rows=1.00 loop...
2	Filter: (name = 'Alice'::text)
3	Rows Removed by Filter: 2
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=2
7	Planning Time: 0.200 ms
8	Execution Time: 0.052 ms

CREATE INDEX idx_users_name ON users (name);

EXPLAIN ANALYZE SELECT * FROM users WHERE name = 'Alice';

Data Output Messages Notifications

Showing rov

	QUERY PLAN text
1	Seq Scan on users (cost=0.00..1.04 rows=1 width=68) (actual time=0.043..0.056 rows=1.00 loop...
2	Filter: (name = 'Alice'::text)
3	Rows Removed by Filter: 2
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=15 read=1
7	Planning Time: 2.881 ms
8	Execution Time: 0.081 ms

K. J. Somaiya School of Engineering, Mumbai-77

(Somaiya Vidyavihar University)

Department of Computer Engineering

Post Lab Question:

1. How does a B-tree differ from a B+-tree? Why is a B+-tree usually preferred as an access structure to a data file?

A B-tree stores data in all nodes, while a B+-tree stores data only in leaf nodes. B+-trees support faster range queries and better sequential access, making them more efficient and commonly used in databases.

2. Construct the B+ Tree for the given input:
   A PARTS file with Part# as the key field includes records with the following Part# values:20, 11, 14, 25, 30,12, 22, 23, 24
   Suppose that the search field values are inserted in the give order in a B+ tree with order of internal node m=3 and order of leaf node m leaf=2. Show how the tree will expand and what the final tree will look like

Order: internal m=3 → max 2 keys
Leaf m=2 → max 2 values

Insert stepwise (sorted inside nodes):

Insert 20 → [20]
Insert 11 → [11,20]
Insert 14 → split → leaves [11,14] | [20] → root [14]

Insert 25 → [20,25]
Insert 30 → split → [20,25] | [30] → root [14,25]

Insert 12 → goes left → [11,12,14] → split → [11,12] | [14] → root split

New root: [14]
Children: [12] and [25]

Insert 22 → middle → [20,22,25] → split

Insert 23 → adjust
Insert 24 → split again

Final B+ Tree:

Root: [14 | 23]

Level 1:
[12] [20] [25]

Leaves:
[11,12] → [14] → [20,22] → [23,24] → [25,30]

Conclusion:

The experiment successfully demonstrated database creation, DML operations, joins, indexing, and B+ tree concepts, improving understanding of efficient data storage, retrieval, and performance optimization in relational database systems.